

Celeste — Server-Side Setup Guide

This document covers the self-hosted infrastructure that powers Celeste's voice, translation, web search, and knowledge-base features. It assumes a Proxmox VE host with LXC containers, but the individual services (Ollama, the TTS gateway, the RAG service) can run on any Linux box with the right dependencies.

Status: Living document — updated as features are added. Current as of the RAG knowledge-base work.

Architecture Overview

Celeste is deliberately split across multiple independent pieces rather than one monolithic server:

Component	Purpose	Typical host
Ollama server(s)	Runs local LLMs (chat + embeddings)	Any machine with a GPU (AMD via ROCm or NVIDIA via CUDA)
TTS Gateway (<code>tts_server.py</code>)	Speech-to-text (Whisper), text-to-speech (Kokoro), translation orchestration	A machine with an NVIDIA GPU (CUDA required for Whisper)
RAG service (<code>rag_server.py</code>)	Document ingestion + semantic search for the knowledge-base feature	Any machine that can reach an Ollama server for embeddings
SearXNG	Self-hosted web search backend	Any lightweight machine/container

None of these require the others to be on the same machine — Celeste's Android app talks to each one independently, and personas can mix and match (e.g., an Ollama server for chat + a separate gateway for voice).

1. NVIDIA GPU Setup (for the TTS Gateway)

If your TTS Gateway runs Whisper/Kokoro on an NVIDIA GPU, you'll need the proprietary driver installed cleanly.

Driver install

```
apt update
```

```
apt install -y pve-headers-$(uname -r) build-essential dkms
```

```
cd /root
```

```
wget
```

```
https://us.download.nvidia.com/XFree86/Linux-x86_64/<VERSION>/NVIDIA-Linux-x86_64-<VERSION>.run
```

```
chmod +x NVIDIA-Linux-x86_64-<VERSION>.run
```

```
./NVIDIA-Linux-x86_64-<VERSION>.run --dkms
```

```
reboot
```

Verify:

```
nvidia-smi
```

⚠ Known gotcha: `nvidia_uvm` not auto-loading

CUDA workloads (PyTorch, ctranslate2) require the `nvidia_uvm` kernel module specifically — `nvidia-smi` can report a perfectly healthy GPU even when this module isn't loaded, since `nvidia-smi` only needs the basic query devices. If CUDA initialization fails with a vague error despite `nvidia-smi` looking fine:

```
lsmod | grep nvidia # check if nvidia_uvm is missing
```

```
modprobe nvidia_uvm # load it manually
```

Make it permanent:

```
echo "nvidia_uvm" >> /etc/modules-load.d/nvidia.conf
```

⚠ Known gotcha: suspend/resume corrupts the GPU driver state

NVIDIA's proprietary Linux driver does not reliably survive `systemctl suspend`, especially when paired with LXC device passthrough. Symptoms: `torch.AcceleratorError: CUDA error: unspecified launch failure`, and `dmesg` showing `NVRM: Xid ... 154, GPU recovery action changed ... Node Reboot Required`. **A full host reboot is required to clear this fault** — a service restart alone will not fix it.

If this GPU host doesn't need to sleep, disable suspend entirely rather than fighting the driver:

```
systemctl mask sleep.target suspend.target hibernate.target hybrid-sleep.target
```

For a machine you do want to sleep sometimes, a clean `poweroff` + cold boot avoids this failure mode entirely (no driver state to corrupt), unlike suspend. Consider setting up Wake-on-LAN if you go this route.

2. LXC Container GPU Passthrough (Proxmox)

If running the TTS Gateway inside an LXC container (recommended for isolation), the container needs the host's NVIDIA device nodes bind-mounted in.

On the host: create device nodes

A systemd service ensures `/dev/nvidia*` nodes exist at boot (they aren't always created automatically on a headless server):

```
/usr/local/bin/nvidia-devices-setup.sh:
```

```
#!/bin/bash
```

```
mknod -m 666 /dev/nvidia0 c 195 0 2>/dev/null || true
```

```
mknod -m 666 /dev/nvidiactl c 195 255 2>/dev/null || true
```

```
mknod -m 666 /dev/nvidia-modeset c 195 254 2>/dev/null || true
```

```
mknod -m 666 /dev/nvidia-uvdm c <UVM_MAJOR> 0 2>/dev/null || true
```

```
mknod -m 666 /dev/nvidia-uvdm-tools c <UVM_MAJOR> 1 2>/dev/null || true
```

```
mkdir -p /dev/nvidia-caps
```

```
mknod -m 444 /dev/nvidia-caps/nvidia-cap1 c <CAPS_MAJOR> 1 2>/dev/null || true
```

```
mknod -m 444 /dev/nvidia-caps/nvidia-cap2 c <CAPS_MAJOR> 2 2>/dev/null || true
```

```
exit 0
```

⚠ Major device numbers for `nvidia-uvm` and `nvidia-caps` are dynamically assigned and can change after any driver reinstall or kernel upgrade. Always check the current numbers before trusting this script:

```
cat /proc/devices | grep nvidia
```

Enable as a service:

```
cat > /etc/systemd/system/nvidia-devices.service << 'EOF'
```

```
[Unit]
```

```
Description=Create Nvidia device nodes
```

```
After=local-fs.target
```

```
[Service]
```

```
Type=oneshot
```

```
ExecStart=/usr/local/bin/nvidia-devices-setup.sh
```

```
RemainAfterExit=yes
```

```
[Install]
```

```
WantedBy=multi-user.target
```

```
EOF
```

```
systemctl enable --now nvidia-devices.service
```

In the container's config (`/etc/pve/lxc/<ID>.conf`)

```
lxc.cgroup2.devices.allow: c 195:* rwm
```

```
lxc.cgroup2.devices.allow: c <UVM_MAJOR>:* rwm
```

```
lxc.cgroup2.devices.allow: c <CAPS_MAJOR>:* rwm
```

```
lxc.mount.entry: /dev/nvidia0 dev/nvidia0 none bind,optional,create=file
```

```
lxc.mount.entry: /dev/nvidiactl dev/nvidiactl none bind,optional,create=file
```

```
lxc.mount.entry: /dev/nvidia-modeset dev/nvidia-modeset none bind,optional,create=file
```

```
lxc.mount.entry: /dev/nvidia-uvm dev/nvidia-uvm none bind,optional,create=file
```

```
lxc.mount.entry: /dev/nvidia-uvm-tools dev/nvidia-uvm-tools none bind,optional,create=file
```

```
lxc.mount.entry: /dev/nvidia-caps dev/nvidia-caps none bind,optional,create=dir
```

Inside the container: userspace libraries only

The container needs the NVIDIA *userspace* libraries (not the kernel module — that stays on the host):

```
./NVIDIA-Linux-x86_64-<VERSION>.run --no-kernel-module
```

```
nvidia-smi # should show the GPU from inside the container
```

3. TTS Gateway ([tts_server.py](#))

FastAPI service providing STT (Whisper), TTS (Kokoro), and translation.

Dependencies

```
python3 -m venv /opt/kokoro-venv
```

```
source /opt/kokoro-venv/bin/activate
```

```
pip install fastapi uvicorn kokoro faster-whisper soundfile numpy requests
```

```
# CUDA-enabled torch (match to your driver's CUDA version):
```

```
pip install torch torchaudio --index-url https://download.pytorch.org/whl/cu130
```

Key configuration notes

- **Credentials:** set your own `GATEWAY_USERNAME/GATEWAY_PASSWORD` in the script — never leave placeholders in a public repo, and rotate immediately if a real password is ever accidentally shared/committed.
- **DEFAULT_MODEL:** pick a generic model available on your Ollama server (e.g. `gemma4:e4b`) — don't hardcode a personal/unusual model name here, since it's the fallback for any request that doesn't specify one.
- **LANG_CONFIG:** maps supported languages to Kokoro voices. Only languages Kokoro actually supports should appear here.
- **UNSUPPORTED_TTS_LANGUAGES:** any language *not* in `LANG_CONFIG` should be listed here — this ensures unsupported languages get a clean "text only" response instead of a Kokoro voice mispronouncing them (or worse, spelling out each letter).

systemd service

[Service]

```
ExecStart=/opt/kokoro-venv/bin/uvicorn tts_server:app --host 0.0.0.0 --port 8880 --ssl-keyfile /root/key.pem --ssl-certfile /root/cert.pem
```

```
Restart=on-failure
```

```
RestartSec=2
```

Self-signed certificate note

The app connects via TLS with a self-signed cert (Trust-On-First-Use pinning) — this is fine for your own gateway, but **never use the same TOFU-pinning HTTP client for calls to public cloud APIs** (Anthropic, OpenAI, etc.), which have real CA-signed certificates and should use standard validation.

4. Ollama Server(s)

Standard Ollama install on any machine with a GPU:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Keeping models warm

By default, Ollama unloads a model from VRAM after 5 minutes of inactivity, causing a real reload delay (tens of seconds) on the next request. To keep a specific model resident longer without pinning every model forever (which risks VRAM contention if you run multiple models):

```
systemctl edit ollama.service
```

```
[Service]
```

```
Environment="OLLAMA_KEEP_ALIVE=30m"
```

Embedding models (for RAG)

```
ollama pull nomic-embed-text
```

5. RAG Knowledge-Base Service ([rag_server.py](#))

Standalone microservice for document ingestion and semantic search, used by Celeste's knowledge-base feature. Unlike the CSM experiment, this service is called **directly by the Android app** (not just internally by the TTS gateway), so it needs to be reachable over the network and properly secured — not bound to loopback only.

Dependencies

```
python3 -m venv /root/rag-server/.venv
```

```
source /root/rag-server/.venv/bin/activate
```

```
pip install fastapi uvicorn chromadb requests python-multipart
```

Embedding model

Reuses your existing Ollama infrastructure — no separate embedding server needed:

```
ollama pull nomic-embed-text
```

Point `OLLAMA_URL` in [rag_server.py](#) at whichever Ollama server has this model pulled.

Chunking strategy

Documents are split sentence-aware, not by raw character count — sentences are grouped into ~500-character chunks without ever cutting mid-sentence. A naive raw-character split (cutting exactly every 500 characters regardless of word/sentence boundaries) noticeably hurt retrieval quality in testing; the sentence-aware approach is the recommended default.

Storage

ChromaDB persists to disk automatically (`chromadb.PersistentClient(path=...)`) — no separate database server needed. To fully reset the knowledge base:

```
systemctl stop rag-server.service
```

```
rm -rf /root/rag-server/chroma_db
```

```
systemctl start rag-server.service
```

Security — required, not optional

This service accepts document uploads and **destructive deletes** with no protection by default. Before exposing it on your network at all:

1. **HTTPS** — reuse the same self-signed cert/key already generated for `tts_server.py` (same host, same cert is fine):

```
ExecStart=/root/rag-server/.venv/bin/uvicorn rag_server:app --host 0.0.0.0 --port 8882  
--ssl-keyfile /root/key.pem --ssl-certfile /root/cert.pem
```

2. **HTTP Basic Auth** — same `HTTPBasicCredentials` pattern as `tts_server.py`. Set your own `GATEWAY_USERNAME/GATEWAY_PASSWORD` in the script (these can be the *same* values as your TTS gateway's credentials, or different — the app stores RAG credentials separately either way, see below).
3. **Bind to 0.0.0.0**, not `127.0.0.1` — this service must be reachable from the phone directly, unlike the CSM microservice pattern (which was internal-only, called only by `tts_server.py` on the same machine).

systemd service

[Service]

```
WorkingDirectory=/root/rag-server
```

```
ExecStart=/root/rag-server/.venv/bin/uvicorn rag_server:app --host 0.0.0.0 --port 8882
--ssl-keyfile /root/key.pem --ssl-certfile /root/cert.pem
```

Restart=on-failure

RestartSec=5

Web admin UI

The service serves a lightweight upload/delete interface directly at its root URL (<https://<server-ip>:8882/>) — open it in any browser to manage documents without needing `curl`. Protected by the same Basic Auth as the API itself.

API endpoints

Endpoint	Purpose
<code>POST /ingest</code>	Add pre-extracted text (used by the Android app, which extracts text client-side first)
<code>POST /ingest-file</code>	Upload a raw file directly (used by the web admin UI — server extracts text)
<code>POST /retrieve</code>	Semantic search — returns top-k relevant chunks for a query
<code>GET /sources</code>	List all documents currently stored, with chunk counts
<code>POST /delete</code>	Remove all chunks belonging to a given <code>source_name</code>
<code>GET /health</code>	Service status + total chunk count

App-side configuration

Celeste connects to this service via the **TOFU-pinning client** (same as the TTS gateway) — never the standard/CA-validating client, since this uses a self-signed certificate. RAG credentials are stored **separately** from gateway credentials in the app (dedicated `ragUsername/ragPassword` fields in Settings → Servers → Knowledge Base) — matching by URL to a gateway entry was tried initially and doesn't work reliably, since the RAG service typically runs on a different port than the TTS gateway even on the same host.

6. Security Checklist

- ☐ Gateway credentials are unique, not left as placeholders, rotated if ever shared in plaintext (chat logs, screenshots, etc.)
 - ☐ Cloud provider API calls (Anthropic, OpenAI, Google, DeepSeek) use standard TLS validation — **never** the TOFU-pinning client meant for self-hosted servers
 - ☐ All internal services (RAG, CSM if used) bound to `127.0.0.1`, not `0.0.0.0`
 - ☐ No conversation content or credentials logged unconditionally in release builds — gate behind `BuildConfig.DEBUG`
 - ☐ `.gitignore` excludes signing keys (`*.jks`), `local.properties`, and build artifacts before any public push
-

Troubleshooting Quick Reference

Symptom	Likely cause	Fix
<code>nvidia-smi</code> works but CUDA apps fail with vague errors	<code>nvidia_uvm</code> not loaded	<code>modprobe nvidia_uvm</code> , add to <code>/etc/modules-load.d/</code>
<code>Xid 154</code> in <code>dmesg</code> , CUDA <code>unspecified launch failure</code>	Suspend/resume corrupted driver state	Full host reboot; disable suspend going forward
Container's <code>nvidia-smi</code> fails after a driver reinstall/kernel upgrade	Major device numbers shifted	<code>cat /proc/devices grep nvidia</code> , update LXC config + device script to match
Gateway returns letter-by-letter mispronunciation	Language missing from <code>LANG_CONFIG</code> or <code>UNSUPPORTED_TTS_LANGUAGES</code>	Add the language to the correct list
RAG uploads/queries return 401 despite "correct" credentials	Placeholder credentials (<code>CHANGE_ME_USERNAME/PASSWORD</code>) never actually replaced, or service not restarted after editing	Edit the real values in <code>rag_server.py</code> , then <code>systemctl restart rag-server.service</code>

Symptom	Likely cause	Fix
RAG chunk count looks way too high for one small file	Document uses single line breaks, not blank-line paragraphs, and old raw-character chunking was still in use	Confirm sentence-aware chunking is in place (see §5); re-ingest after upgrading